Device Settings (READ_SETTING_COMMAND)

What Are Device Settings?

Device settings control how your T3000 panel operates, including network configuration, communication ports, display options, and more. These settings are stored in a 400-byte structure on the device.

Where to Access Settings

Web Interface:

- Navigate to: `http://localhost:3003/#/t3000/settings`
- View and edit all device configuration
- Changes are sent to the device automatically

API Endpoint:

- Command: `READ_SETTING_COMMAND`
- Returns: 400-byte array with all settings
- Format: Binary data that needs parsing

Common Settings You Can Configure

Network Settings:

- IP Address, Subnet Mask, Gateway
- DHCP or Static IP
- MAC Address (read-only)

Communication Ports:

- COM0, COM1, COM2 configuration
- Baud rates (9600 to 115200)
- Parity and stop bits

Device Identity:

- Panel Name (up to 20 characters)

- Panel Number
- Serial Number
- Device Instance ID

Time Synchronization:

- Time Zone
- NTP Server address
- Auto sync with PC or NTP server

Display Options:

- LCD always on/off
- Panel type
- Customer units

Example: Changing IP Address

What you see in the interface:

```
TEXT

IP Address: 192.168.0.144
Subnet: 255.255.255.0
Gateway: 192.168.0.1
```

How it's stored (first 12 bytes):

```
TEXT

[192, 168, 0, 144,    // IP address
 255, 255, 255, 0,    // Subnet mask
 192, 168, 0, 1,      // Gateway
 ...]
```

Binary Data Structure

The device settings are stored as a C/C++ union structure with 400 bytes total. The data can be accessed either as raw bytes (`all[400]`) or as a structured format (`reg`).

C++ Structure Definition

C++

```
typedef union {
    uint8_t all[400];
    struct {
        // Network Configuration (Bytes 0-18)
        uint8_t ip_addr[4];          // Bytes 0-3
        uint8_t subnet[4];           // Bytes 4-7
        uint8_t gate_addr[4];        // Bytes 8-11
        uint8_t mac_addr[6];         // Bytes 12-17
        uint8_t tcp_type;            // Byte 18: 0=DHCP, 1=Static
        uint8_t mini_type;           // Byte 19
        uint8_t debug;               // Byte 20

        // Protocol Info (Bytes 21-37)
        Str_Pro_Info pro_info;       // 17 bytes (21-37)

        // COM Port Configuration (Bytes 38-43)
        uint8_t com0_config;         // Byte 38
        uint8_t com1_config;         // Byte 39
        uint8_t com2_config;         // Byte 40
        uint8_t refresh_flash_timer; // Byte 41
        uint8_t en_plug_n_play;      // Byte 42
        uint8_t reset_default;       // Byte 43: 88=reset, 77=identify
        uint8_t com_baudrate0;       // Byte 44
        uint8_t com_baudrate1;       // Byte 45
        uint8_t com_baudrate2;       // Byte 46

        // User Settings (Bytes 47-51)
        uint8_t user_name;           // Byte 47: 0=no, 1=disable, 2=enable
        uint8_t custmer_unite;       // Byte 48: 0=no
        uint8_t usb_mode;            // Byte 49: 0=device, 1=host
        uint8_t network_number;      // Byte 50
        uint8_t panel_type;          // Byte 51

        // Panel Name (Bytes 52-71)
        char panel_name[20];         // Bytes 52-71

        // Panel Configuration (Bytes 72-73)
        uint8_t en_panel_name;       // Byte 72
```

```
uint8_t panel_number;         // Byte 73

// DynDNS Configuration (Bytes 74-169)
char dyndns_user[DYNDNS_MAX_USERNAME_SIZE];
char dyndns_pass[DYNDNS_MAX_PASSWORD_SIZE];
char dyndns_domain[DYNDNS_MAX_DOMAIN_SIZE];
uint8_t en_dyndns;            // 0=no, 1=disable, 2=enable
uint8_t dyndns_provider;      // 0=3322.org, 1=dyndns.com, 2=no-ip.com
uint16_t dyndns_update_time;  // Minutes

// Time Configuration
uint8_t en_sntp;              // 0=no, 1=disable
signed short time_zone;       // Timezone offset
unsigned int n_serial_number; // Device serial number
UN_Time update_dyndns;

// Advanced Settings
uint16_t mstp_network_number;
uint8_t BBMD_EN;
uint8_t sd_exist;             // 1=no, 2=yes, 3=format error
unsigned short modbus_port;
unsigned char modbus_id;
unsigned int object_instance;
unsigned int time_update_since_1970;
unsigned char time_zone_summer_daytime;
char sntp_server[30];         // NTP server address

// Device Capabilities
unsigned char zegbee_exsit;
unsigned char LCD_Display;    // 1=always on, 0=off
unsigned char flag_time_sync_pc;
unsigned char time_sync_auto_manual; // 0=NTP, 1=PC sync
unsigned char sync_time_results;     // 0=failed, 1=success
unsigned char mstp_id;
unsigned short zigbee_panid;
unsigned char max_master;            // Max 245
unsigned char special_flag;          // bit0=PT1K, bit1=PT100
unsigned char uart_parity[3];
unsigned char uart_stopbit[3];

lcdconfig display_lcd;
```

```
        unsigned char start_month;
        unsigned char start_day;
        unsigned char end_month;
        unsigned char end_day;
        unsigned char network_number_hi;
        unsigned char webview_json_flash;    // 0=old, 2=new JSON
        unsigned char max_var;               // ESP32 only, ST fixed 128
        unsigned char max_in;                // ESP32 only, ST fixed 64
        unsigned char max_out;               // ESP32 only, ST fixed 64
        unsigned char fix_com_config;        // 0=auto, non-0=fixed
        unsigned char write_flash;           // Minutes, 0=disabled
    } reg;
} Str_Setting_Info;
```

TypeScript/JavaScript Parser

Complete Parsing Implementation

TYPESCRIPT

```typescript
interface DeviceSettings {
  // Network
  ipAddress: string;
  subnet: string;
  gateway: string;
  macAddress: string;
  tcpType: 'DHCP' | 'Static';

  // COM Ports
  com0: {
    config: number;
    baudrate: number;
    parity: number;
    stopbit: number;
  };
  com1: {
    config: number;
    baudrate: number;
    parity: number;
    stopbit: number;
  };
  com2: {
    config: number;
    baudrate: number;
    parity: number;
    stopbit: number;
  };

  // Panel Info
  panelName: string;
  panelNumber: number;
  panelType: number;
  serialNumber: number;

  // BACnet
  networkNumber: number;
```

```
  objectInstance: number;
  mstpId: number;

  // Time Settings
  timeZone: number;
  sntpServer: string;
  enableSntp: boolean;
  timeSyncMode: 'NTP' | 'PC';

  // Modbus
  modbusPort: number;
  modbusId: number;

  // Display
  lcdDisplay: boolean;

  // Advanced
  usbMode: 'Device' | 'Host';
  debugMode: boolean;
  plugAndPlay: boolean;
  sdCardExists: boolean;

  // Limits (ESP32)
  maxVar: number;
  maxIn: number;
  maxOut: number;
}

/**
 * Parse 400-byte settings array into DeviceSettings object
 */
export function parseDeviceSettings(data: number[]): DeviceSettings {
  if (data.length !== 400) {
    throw new Error(`Invalid data length: ${data.length}, expected 400 bytes`);
  }

  return {
    // Network Configuration (Bytes 0-18)
    ipAddress: `${data[0]}.${data[1]}.${data[2]}.${data[3]}`,
    subnet: `${data[4]}.${data[5]}.${data[6]}.${data[7]}`,
    gateway: `${data[8]}.${data[9]}.${data[10]}.${data[11]}`,
```

```
  macAddress: [12, 13, 14, 15, 16, 17]
    .map(i => data[i].toString(16).padStart(2, '0').toUpperCase())
    .join(':'),
  tcpType: data[18] === 0 ? 'DHCP' : 'Static',

  // COM Port Configuration
  com0: {
    config: data[38],
    baudrate: parseBaudRate(data[44]),
    parity: data[260],  // uart_parity[0]
    stopbit: data[263]  // uart_stopbit[0]
  },
  com1: {
    config: data[39],
    baudrate: parseBaudRate(data[45]),
    parity: data[261],  // uart_parity[1]
    stopbit: data[264]  // uart_stopbit[1]
  },
  com2: {
    config: data[40],
    baudrate: parseBaudRate(data[46]),
    parity: data[262],  // uart_parity[2]
    stopbit: data[265]  // uart_stopbit[2]
  },

  // Panel Information (Bytes 52-73)
  panelName: parseString(data, 52, 20),
  panelNumber: data[73],
  panelType: data[51],

  // Serial Number (4 bytes, little-endian at byte 181)
  serialNumber:
    data[181] |
    (data[182] << 8) |
    (data[183] << 16) |
    (data[184] << 24),

  // Network
  networkNumber: data[50],
  objectInstance:
    data[198] |
```

```
      (data[199] << 8) |
      (data[200] << 16) |
      (data[201] << 24),
    mstpId: data[244],

    // Time Configuration
    timeZone: (data[180] | (data[181] << 8)) as number,  // signed short
    sntpServer: parseString(data, 208, 30),
    enableSntp: data[179] === 2,
    timeSyncMode: data[241] === 0 ? 'NTP' : 'PC',

    // Modbus
    modbusPort: data[194] | (data[195] << 8),
    modbusId: data[196],

    // Display
    lcdDisplay: data[239] === 1,

    // Advanced
    usbMode: data[49] === 0 ? 'Device' : 'Host',
    debugMode: data[20] === 1,
    plugAndPlay: data[42] === 1,
    sdCardExists: data[193] === 2,

    // ESP32 Limits
    maxVar: data[267],
    maxIn: data[268],
    maxOut: data[269]
  };
}

/**
 * Helper: Parse null-terminated string from byte array
 */
function parseString(data: number[], offset: number, maxLength: number): string {
  const bytes = [];
  for (let i = 0; i < maxLength; i++) {
    const byte = data[offset + i];
    if (byte === 0) break;  // Null terminator
    bytes.push(byte);
  }
```

```
  return String.fromCharCode(...bytes);
}

/**
 * Helper: Convert baudrate code to actual value
 */
function parseBaudRate(code: number): number {
  const rates = [9600, 19200, 38400, 57600, 115200];
  return rates[code] || 9600;
}

/**
 * Helper: Convert settings object back to 400-byte array
 */
export function encodeDeviceSettings(settings: Partial<DeviceSettings>, original: number
  const data = [...original];  // Clone original array

  // Update IP address if changed
  if (settings.ipAddress) {
    const parts = settings.ipAddress.split('.').map(Number);
    data[0] = parts[0];
    data[1] = parts[1];
    data[2] = parts[2];
    data[3] = parts[3];
  }

  // Update subnet if changed
  if (settings.subnet) {
    const parts = settings.subnet.split('.').map(Number);
    data[4] = parts[0];
    data[5] = parts[1];
    data[6] = parts[2];
    data[7] = parts[3];
  }

  // Update gateway if changed
  if (settings.gateway) {
    const parts = settings.gateway.split('.').map(Number);
    data[8] = parts[0];
    data[9] = parts[1];
    data[10] = parts[2];
```

```
    data[11] = parts[3];
  }

  // Update TCP type
  if (settings.tcpType !== undefined) {
    data[18] = settings.tcpType === 'DHCP' ? 0 : 1;
  }

  // Update panel name
  if (settings.panelName !== undefined) {
    encodeString(data, 52, settings.panelName, 20);
  }

  // Update NTP server
  if (settings.sntpServer !== undefined) {
    encodeString(data, 208, settings.sntpServer, 30);
  }

  return data;
}

/**
 * Helper: Write string to byte array with null termination
 */
function encodeString(data: number[], offset: number, str: string, maxLength: number): v
  for (let i = 0; i < maxLength; i++) {
    data[offset + i] = i < str.length ? str.charCodeAt(i) : 0;
  }
}
```

Usage Examples

Example 1: Parse Sample Data

TYPESCRIPT

```typescript
// Sample 400-byte array from device
const sampleData = [
  192, 168, 0, 144,      // IP: 192.168.0.144
  255, 255, 255, 0,      // Subnet: 255.255.255.0
  192, 168, 0, 0,        // Gateway: 192.168.0.0
  0, 14, 198, 242, 224, 195,  // MAC: 00:0E:C6:F2:E0:C3
  0,                     // TCP type: DHCP
  5,                     // Mini type
  0,                     // Debug off
  // ... rest of 400 bytes
];

// Parse the data
const settings = parseDeviceSettings(sampleData);

console.log('Network Configuration:');
console.log(`  IP Address: ${settings.ipAddress}`);
console.log(`  Subnet: ${settings.subnet}`);
console.log(`  Gateway: ${settings.gateway}`);
console.log(`  MAC Address: ${settings.macAddress}`);
console.log(`  Connection Type: ${settings.tcpType}`);

console.log('\nDevice Information:');
console.log(`  Panel Name: ${settings.panelName}`);
console.log(`  Serial Number: ${settings.serialNumber}`);
console.log(`  Panel Number: ${settings.panelNumber}`);

console.log('\nCommunication:');
console.log(`  COM0 Baudrate: ${settings.com0.baudrate}`);
console.log(`  Modbus Port: ${settings.modbusPort}`);
console.log(`  Modbus ID: ${settings.modbusId}`);

console.log('\nTime Settings:');
console.log(`  NTP Server: ${settings.sntpServer}`);
console.log(`  Time Sync: ${settings.timeSyncMode}`);
```

Expected Output:

TEXT

```
Network Configuration:
  IP Address: 192.168.0.144
  Subnet: 255.255.255.0
  Gateway: 192.168.0.0
  MAC Address: 00:0E:C6:F2:E0:C3
  Connection Type: DHCP

Device Information:
  Panel Name: Fandu144-BB
  Serial Number: 1581674
  Panel Number: 144

Communication:
  COM0 Baudrate: 19200
  Modbus Port: 502
  Modbus ID: 1

Time Settings:
  NTP Server: nz.pool.ntp.org
  Time Sync: NTP
```

Example 2: Update Settings

TYPESCRIPT

```typescript
// Read current settings
const currentData = await fetchDeviceSettings(deviceId);
const currentSettings = parseDeviceSettings(currentData);

// Modify specific fields
const updatedSettings = {
  ...currentSettings,
  ipAddress: '192.168.1.100',
  subnet: '255.255.255.0',
  gateway: '192.168.1.1',
  tcpType: 'Static' as const,
  panelName: 'New Panel Name'
};

// Encode back to binary
const updatedData = encodeDeviceSettings(updatedSettings, currentData);

// Send to device
await saveDeviceSettings(deviceId, updatedData);
```

Example 3: React Component

TYPESCRIPT

```typescript
import { useState, useEffect } from 'react';
import { parseDeviceSettings, type DeviceSettings } from './settingsParser';

export function SettingsPage({ deviceId }: { deviceId: number }) {
  const [settings, setSettings] = useState<DeviceSettings | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    loadSettings();
  }, [deviceId]);

  async function loadSettings() {
    try {
      const response = await fetch(`/api/device/${deviceId}/settings`);
      const data = await response.json();
      const parsed = parseDeviceSettings(data.settings);
      setSettings(parsed);
    } catch (error) {
      console.error('Failed to load settings:', error);
    } finally {
      setLoading(false);
    }
  }

  async function saveSettings() {
    if (!settings) return;

    try {
      const response = await fetch(`/api/device/${deviceId}/settings`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(settings)
      });

      if (response.ok) {
        alert('Settings saved successfully!');
      }
    } catch (error) {
```

```
      console.error('Failed to save settings:', error);
    }
  }

  if (loading) return <div>Loading...</div>;
  if (!settings) return <div>Failed to load settings</div>;

  return (
    <div>
      <h1>Device Settings</h1>

      <section>
        <h2>Network Configuration</h2>
        <label>
          IP Address:
          <input
            value={settings.ipAddress}
            onChange={(e) => setSettings({
              ...settings,
              ipAddress: e.target.value
            })}
          />
        </label>
        <label>
          Connection Type:
          <select
            value={settings.tcpType}
            onChange={(e) => setSettings({
              ...settings,
              tcpType: e.target.value as 'DHCP' | 'Static'
            })}
          >
            <option value="DHCP">DHCP</option>
            <option value="Static">Static</option>
          </select>
        </label>
      </section>

      <section>
        <h2>Device Information</h2>
        <label>
```

```
          Panel Name:
          <input
            value={settings.panelName}
            maxLength={20}
            onChange={(e) => setSettings({
              ...settings,
              panelName: e.target.value
            })}
          />
        </label>
        <p>Serial Number: {settings.serialNumber}</p>
        <p>MAC Address: {settings.macAddress}</p>
      </section>

      <button onClick={saveSettings}>Save Settings</button>
    </div>
  );
}
```

Byte Offset Reference Table

Quick lookup for field locations in the 400-byte array:

Offset	Size	Field	Format	Example
0-3	4	IP Address	IPv4	`[192,168,0,144]` = 192.168.0.144
4-7	4	Subnet Mask	IPv4	`[255,255,255,0]`
8-11	4	Gateway	IPv4	`[192,168,0,1]`
12-17	6	MAC Address	Hex	`[00,0E,C6,F2,E0,C3]`
18	1	TCP Type	0=DHCP, 1=Static	`0`
19	1	Mini Type	Module Number	`5` = T3-BB (see below)
20	1	Debug Mode	Boolean	`0`
38	1	COM0 Config	Uint8	`1`
39	1	COM1 Config	Uint8	`0`
40	1	COM2 Config	Uint8	`0`
44	1	COM0 Baudrate	Index	`1` = 19200
45	1	COM1 Baudrate	Index	`0` = 9600
46	1	COM2 Baudrate	Index	`9` = 115200
49	1	USB Mode	0=Device, 1=Host	`0`
50	1	Network Number	Uint8	`255`
51	1	Panel Type	Uint8	`74`
52-71	20	Panel Name	String	`"Fandu144-BB"`
73	1	Panel Number	Uint8	`144`
181-184	4	Serial Number	Uint32 LE	`1581674`
194-195	2	Modbus Port	Uint16 LE	`502`
196	1	Modbus ID	Uint8	`1`
197-200	4	Object Instance	Uint32 LE	`246`
208-237	30	SNTP Server	String	`"nz.pool.ntp.org"`
239	1	LCD Display	1=On, 0=Off	`0`
240	1	Time Sync Flag	Boolean	`1`

Offset	Size	Field	Format	Example
241	1	Sync Mode	0=NTP, 1=PC	`1`
244	1	MSTP ID	Uint8	`0`
260-262	3	UART Parity	Uint8[3]	`[0,0,0]`
263-265	3	UART Stop Bit	Uint8[3]	`[0,0,0]`
267	1	Max Variables	Uint8	`255` (or 128)
268	1	Max Inputs	Uint8	`255` (or 64)
269	1	Max Outputs	Uint8	`255` (or 64)

Mini Type (Byte 19) - Module Number Mapping

The **mini_type** field at byte 19 identifies the hardware module type. This maps to user-friendly product names:

Value	Constant	Product Name	Description
0	PRODUCT_CM5	CM5	Original CM5 controller
1	BIG_MINIPANEL	T3-BB (Asix)	BACnet Building Controller (Asix chip)
2	SMALL_MINIPANEL	T3-LB (Asix)	LON Building Controller (Asix chip)
3	TINY_MINIPANEL	T3-TB (Asix)	Tiny Building Controller (Asix chip)
4	TINY_EX_MINIPANEL	T3-TB (Asix)	Tiny Building Controller Extended (Asix chip)
5	MINIPANELARM	**T3-BB**	BACnet Building Controller (ARM chip)
6	MINIPANELARM_LB	**T3-LB**	LON Building Controller (ARM chip)
7	MINIPANELARM_TB	**T3-TB**	Tiny Building Controller (ARM chip)
8	MINIPANELARM_NB	**T3-Nano**	Nano Building Controller
9	T3_TSTAT10	**TSTAT10**	Thermostat
11	T3_OEM	**T3-OEM**	OEM Controller
12	T3_TB_11I	**T3-TB-11I**	Tiny Building Controller with 11 inputs
13	T3_FAN_MODULE	**T3-FAN-MODULE**	Fan control module
14	T3_OEM_12I	**T3-OEM-12I**	OEM Controller with 12 inputs
15	T3_AIRLAB	**T3-AIRLAB**	Air quality lab controller
16	T3_ESP_TRANSDUCER	**T3-ESP-TRANSDUCER**	ESP32-based transducer
17	T3_ESP_TSTAT9	**T3-ESP-TSTAT9**	ESP32-based thermostat
18	T3_ESP_SAUTER	**T3-ESP-SAUTER**	ESP32-based Sauter integration
19	T3_ESP_RMC	**T3-RMC**	Room Management Controller
21	T3_ESP_LW	**T3-ESP-LW**	ESP32-based lighting controller
22	T3_NG2_TYPE2	**T3-NG2**	Next-gen controller type 2
26	T3_3IIC	**T3-3IIC**	3-channel IIC controller
43	PID_T322AI	**T322AI**	22-channel analog input
44	T38AI8AO6DO	**T38AI8AO6DO**	8 AI, 8 AO, 6 DO controller

Code Reference: `T3000-Source/T3000/global_define.h` and `T3000-Source/T3000/BacnetSetting.cpp::Getminitypename()`

Usage Example:

```typescript
// Parse mini_type from byte 19
const miniType = data[19];

// Map to friendly name
const moduleNames: { [key: number]: string } = {
  0: 'CM5',
  1: 'T3-BB', 2: 'T3-LB', 3: 'T3-TB', 4: 'T3-TB',
  5: 'T3-BB', 6: 'T3-LB', 7: 'T3-TB', 8: 'T3-Nano',
  9: 'TSTAT10', 11: 'T3-OEM', 12: 'T3-TB-11I',
  13: 'T3-FAN-MODULE', 14: 'T3-OEM-12I', 15: 'T3-AIRLAB',
  16: 'T3-ESP-TRANSDUCER', 17: 'T3-ESP-TSTAT9', 18: 'T3-ESP-SAUTER',
  19: 'T3-RMC', 21: 'T3-ESP-LW', 22: 'T3-NG2',
  26: 'T3-3IIC', 43: 'T322AI', 44: 'T38AI8AO6DO',
};

const moduleName = moduleNames[miniType] || `Unknown (${miniType})`;
console.log(`Module: ${moduleName}`); // "Module: T3-BB"
```

Note: Values 1-4 are legacy Asix-based controllers. Modern ARM-based controllers use values 5-8 and above.

Rust API Endpoint

RUST

```rust
// api/src/device_settings/routes.rs

use axum::{extract::Path, Json};
use crate::error::Result;

#[derive(Serialize, Deserialize)]
pub struct SettingsResponse {
    pub device_id: u32,
    pub settings: Vec<u8>,  // 400 bytes
}

/// GET /api/devices/:id/settings
pub async fn get_device_settings(
    Path(device_id): Path<u32>
) -> Result<Json<SettingsResponse>> {
    // Call C++ FFI to read settings
    let settings_data = read_device_settings_ffi(device_id)?;

    Ok(Json(SettingsResponse {
        device_id,
        settings: settings_data
    }))
}

/// POST /api/devices/:id/settings
pub async fn update_device_settings(
    Path(device_id): Path<u32>,
    Json(data): Json<Vec<u8>>
) -> Result<Json<SettingsResponse>> {
    if data.len() != 400 {
        return Err(Error::InvalidDataLength);
    }

    // Call C++ FFI to write settings
    write_device_settings_ffi(device_id, &data)?;

    Ok(Json(SettingsResponse {
        device_id,
```

```
        settings: data
    }))
}
```

Frontend Service

TYPESCRIPT

```typescript
// src/services/deviceSettingsService.ts

export class DeviceSettingsService {
  private baseUrl = '/api/devices';

  async getSettings(deviceId: number): Promise<DeviceSettings> {
    const response = await fetch(`${this.baseUrl}/${deviceId}/settings`);
    if (!response.ok) throw new Error('Failed to fetch settings');

    const data = await response.json();
    return parseDeviceSettings(data.settings);
  }

  async updateSettings(
    deviceId: number,
    settings: Partial<DeviceSettings>,
    originalData: number[]
  ): Promise<void> {
    const encoded = encodeDeviceSettings(settings, originalData);

    const response = await fetch(`${this.baseUrl}/${deviceId}/settings`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(encoded)
    });

    if (!response.ok) throw new Error('Failed to update settings');
  }
}
```

Common Issues & Solutions

Issue: String Fields Show Garbage Characters

Problem: Panel name or NTP server shows random characters

Solution: Strings are null-terminated. Always check for null byte (0) when parsing:

TYPESCRIPT

```typescript
function parseString(data: number[], offset: number, maxLength: number): string {
  const bytes = [];
  for (let i = 0; i < maxLength; i++) {
    const byte = data[offset + i];
    if (byte === 0) break;  // Stop at null terminator
    if (byte >= 32 && byte <= 126) {  // Printable ASCII only
      bytes.push(byte);
    }
  }
  return String.fromCharCode(...bytes);
}
```

Issue: Wrong Byte Order for Multi-byte Values

Problem: Serial number or object instance shows wrong value

Solution: Most multi-byte integers use **little-endian** format:

TYPESCRIPT

```typescript
// Correct: Little-endian (LSB first)
const serialNumber =
  data[181] |
  (data[182] << 8) |
  (data[183] << 16) |
  (data[184] << 24);

// Wrong: Big-endian
const wrong =
  (data[181] << 24) |
  (data[182] << 16) |
  (data[183] << 8) |
  data[184];
```

Issue: Settings Not Persisting

Problem: Changes revert after device restart

Solution: Check `write_flash` field (byte 270). If 0, settings aren't saved to flash:

```typescript
// Enable flash writing every 5 minutes
data[270] = 5;  // Save to flash every 5 minutes
```

See Also

- Device Configuration Guide
- Control Messages
- BACnet Protocol
- REST API Reference